CLASE PRÁCTICA

Observabilidad

De la señal al diagnóstico: métricas, logs y dashboards

Métricas

Prometheus

Logs

MySQL

Dashboards

Grafana

Made with GAMMA

Observabilidad: entender qué pasa dentro de una aplicación

Una aplicación puede responder correctamente la mayoría de las veces y, aun así, tener problemas difíciles de detectar: algunas rutas se vuelven lentas, aumenta la cantidad de errores o una función deja de comportarse como se esperaba.

La observabilidad consiste en recopilar y analizar información que nos permita entender el estado y el comportamiento de un sistema.

En esta clase vamos a ver dos tipos de información:

- Métricas: números que resumen el comportamiento de la aplicación.
- Logs: mensajes que describen eventos concretos.

Usaremos Prometheus para consultar y almacenar métricas, y Grafana para visualizar métricas y logs.



2 Contributors

Made with GAMMA

¿Qué es una métrica?

Una métrica es un valor numérico que se registra a lo largo del tiempo. Nos ayuda a observar tendencias, comparar períodos y detectar cambios en el comportamiento del sistema.

Ejemplos de métricas:

- Cantidad de solicitudes recibidas.
- Porcentaje de respuestas con error.
- Duración de las solicitudes.
- Uso de CPU o memoria.
- Cantidad de elementos pendientes en una cola.

Una métrica permite responder preguntas como:

- ¿Aumentaron los errores durante los últimos minutos?
- ¿Qué ruta está tardando más?
- ¿Cuántas solicitudes recibe la aplicación por segundo?

Tiempo

eje horizontal

Valor numérico

eje vertical

Tendencia

¿sube o baja?

Made with GAMMA

Tipos de métricas que aparecen en la demo

En la demo usamos dos instrumentos de Prometheus:

Contador (Counter)

Solo aumenta. Sirve para contar sucesos, como las solicitudes atendidas. Por ejemplo, cada respuesta incrementa requests_total.

Histograma (Histogram)

Registra observaciones, como la duración de las solicitudes. Prometheus agrupa esas duraciones en rangos y permite calcular estadísticas, como promedios aproximados o percentiles.

En la demo, las métricas tienen etiquetas de ruta y código HTTP. Así podemos diferenciar, por ejemplo, respuestas 200 de respuestas 500.

Instrumento	Cómo se comporta
Counter requests_total	Sube y nunca baja: 1 → 2 → 3 → 4
Histogram request_duration_seconds	Agrupa duraciones en rangos (cubetas) que se llenan según el tiempo de cada solicitud

Made with GAMMA

¿Qué es un log?

Un log es un registro de un evento que ocurrió en la aplicación. Puede incluir un mensaje, su nivel de importancia y cuándo sucedió.

Ejemplos:

```
info: operacion_exitosa
warn: operacion_lenta
error: operacion_fallida: fallo_simulado
```

Los logs ayudan a investigar situaciones concretas:

- ¿Qué operación falló?
- ¿Qué mensaje produjo el sistema?
- ¿A qué hora ocurrió el evento?
- ¿Fue una advertencia o un error?

En la demo, los logs se escriben en MySQL, en la tabla app_logs.

info	warn	error
operacion_exitosa	operacion_lenta	operacion_fallida

Made with GAMMA

Métricas y logs: se complementan, no son lo mismo

Métrica: resume lo que está ocurriendo y permite ver tendencias y cantidades. Ejemplo: 20 respuestas HTTP 500 en un minuto.

Log: describe un evento concreto y ayuda a investigar el detalle. Ejemplo: operacion_fallida: fallo_simulado.

Si el contador de errores aumenta, la métrica nos avisa que algo merece atención. Luego, los logs pueden ayudarnos a descubrir qué eventos ocurrieron.

Importante: en esta demo, los logs no se guardan en Prometheus. Prometheus guarda métricas; MySQL guarda logs.

Métrica	Log
Prometheus almacena métricas	MySQL almacena logs

Made with GAMMA

CorrelationID o TrackID: seguir una solicitud de principio a fin

Un CorrelationID (también llamado TrackID) es un identificador único que se asigna a una solicitud cuando entra al sistema y se propaga a todos los servicios, funciones y registros que participan en su procesamiento.

Para qué sirve:

- Relacionar logs de distintos servicios que pertenecen a la misma solicitud.
- Reconstruir el recorrido completo de una petición cuando algo falla.
- Diagnosticar errores y latencias en sistemas distribuidos o con microservicios.
- Facilitar el soporte: un usuario puede reportar su ID y el equipo encuentra todos sus eventos.

Ejemplo: la API recibe una solicitud con el encabezado X-Correlation-ID: 7f3a9c. Cada log que genera esa solicitud incluye ese mismo valor, así se puede filtrar por 7f3a9c y ver todo lo que ocurrió.

Buenas prácticas: generar el ID en el primer punto de entrada si el cliente no lo envía, reutilizarlo si ya viene en la solicitud y devolverlo en la respuesta.

Made with GAMMA

¿Qué hace Prometheus?

Prometheus es un sistema de monitoreo y almacenamiento de métricas. En esta demo:

1. La API publica métricas en el endpoint /metrics.
2. Prometheus consulta ese endpoint periódicamente.
3. Almacena los valores a lo largo del tiempo.
4. Permite consultar esos datos, por ejemplo, para ver el total de solicitudes.

Prometheus utiliza normalmente un modelo llamado scraping: va a consultar la aplicación para recoger métricas.

En esta clase, la consulta se configura para repetirse cada cinco segundos.

API

/metrics

Scraping

cada 5 s

Prometheus

almacena métricas

Made with GAMMA

¿Qué hace Grafana?

Grafana permite visualizar información proveniente de fuentes de datos. En la demo consulta:

- Prometheus, para los gráficos de métricas.
- MySQL, para mostrar los logs guardados en app_logs.

Grafana no es quien genera las métricas de la API ni quien decide qué logs escribir. Su función principal aquí es consultar esas fuentes y presentarlas desde una interfaz común.

En el tablero veremos:

- Solicitudes por segundo, separadas por ruta y estado HTTP.
- Duración media por ruta.
- Logs recientes de la aplicación.

Prometheus	**MySQL**	**Grafana**
métricas	logs	dashboard

Made with GAMMA

Recorrido completo de los datos

El flujo de una solicitud es:

Se envía una solicitud → API ejecuta una ruta → genera un log en MySQL y registra duración y estado en /metrics → Prometheus recoge las métricas → Grafana consulta ambas fuentes.

La API produce la información. MySQL y Prometheus la almacenan en formatos distintos. Grafana permite consultarla y verla desde una interfaz común.

1. API

ejecuta una ruta

2. MySQL

guarda logs

3. Prometheus

guarda métricas

4. Grafana

visualiza ambas

Made with GAMMA

La demo de Clase 8: qué vamos a explorar

La demo está en CLASE_8/DEMO.

Es una aplicación web desarrollada en Go que simula tres situaciones:

- /ok: operación exitosa.
- /slow: operación lenta, con una espera de aproximadamente 750 ms.
- /error: fallo controlado que responde con HTTP 500.
- /compra/ok: proceso de compra con recorrido entre "distintas" APIs, con trackID
- /compra/error: proceso de compra con recorrido entre "distintas" APIs + fallo en la "ultima" api, con trackID
- /compra/error-pedidos: proceso de compra con recorrido entre "distintas" APIs + fallo en la api "del medio", con trackID

Además, incluye /health para comprobar que la API está viva y /metrics para exponer métricas a Prometheus.

/ok	**/slow**	**/error**
éxito	≈ 750 ms	HTTP 500

Made with GAMMA

Paso 1: iniciar los servicios

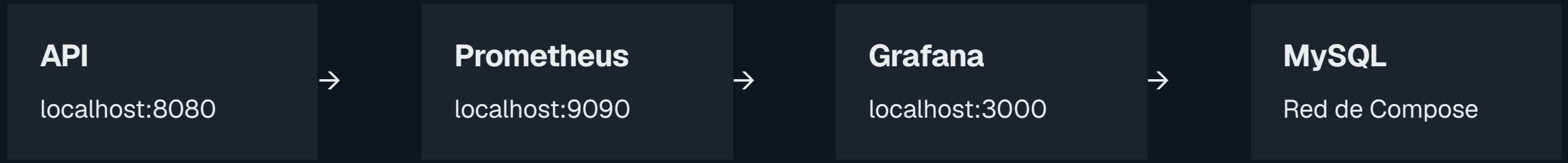
Requisitos: tener Docker Desktop o un servicio Docker activo.

Abrir una terminal en CLASE_8/DEMO y ejecutar:

```
docker compose up --build -d
```

Esto construye la imagen de la API y levanta los servicios de la demo:

- API: http://localhost:8080
- Prometheus: http://localhost:9090
- Grafana: http://localhost:3000
- MySQL: accesible desde los demás contenedores de Compose



La primera vez puede tardar porque Docker debe descargar imágenes y compilar la aplicación.

Made with GAMMA

Paso 2: probar las rutas simples de la API

Enviar solicitudes desde una terminal:

```
curl http://localhost:8080/ok
curl http://localhost:8080/slow
curl http://localhost:8080/error
```

Observar las diferencias:

- /ok devuelve una respuesta exitosa.
- /slow responde después de una demora intencional.
- /error devuelve HTTP 500.

Cada ruta también genera un log en MySQL. Repetir las solicitudes permite acumular datos para observarlos en Prometheus y Grafana.

/ok	**/slow**	**/error**
respuesta exitosa	respuesta con demora	HTTP 500

Made with GAMMA

Paso 3: probar las rutas con Track ID de la API

Enviar solicitudes desde una terminal:

```
curl http://localhost:8080/compra/ok
curl http://localhost:8080/compra/error
curl http://localhost:8080/compra/error-pedidos
```

Made with GAMMA

Paso 4: entender cómo la API registra métricas

El archivo CLASE_8/DEMO/observability/metrics.go define un contador llamado requests_total y un histograma llamado request_duration_seconds.

Un middleware de Gin mide cuánto tarda cada solicitud. Al terminar, registra el contador y la duración usando la ruta y el código HTTP como etiquetas.

El archivo CLASE_8/DEMO/main.go expone las métricas en /metrics. El endpoint se puede probar así:

```
curl http://localhost:8080/metrics
```

La respuesta es texto en un formato que Prometheus puede leer.

Medir

inicio de la solicitud

Ejecutar

la ruta

Registrar

contador y duración

Made with GAMMA

Paso 5: revisar qué recoge Prometheus

La configuración está en CLASE_8/DEMO/prometheus.yml.

Allí se indica que Prometheus consulte app:8080/metrics cada cinco segundos. app es el nombre del servicio de la API dentro de la red de Docker Compose.

Abrir http://localhost:9090/targets. El objetivo llamado demo-app debería aparecer como UP.

Para consultar el contador desde la interfaz de Prometheus, usar:

```
requests_total{job="demo-app"}
```

El contador queda separado por las etiquetas de ruta y estado HTTP.

API /metrics

expone métricas

Prometheus

consulta cada 5 s

Targets

demo-app: UP

Made with GAMMA

Paso 6: revisar logs en MySQL y en Grafana

Cada ruta escribe un mensaje en la tabla app_logs de MySQL. Para consultar las últimas filas desde la terminal:

```
docker compose exec mysql mysql -u demo -pdemo observabilidad -e "SELECT * FROM app_logs ORDER BY id DESC LIMIT 10;"
```

Abrir Grafana en http://localhost:3000.

Credenciales de esta demo local: usuario admin, clave admin.

El dashboard Demo de observabilidad muestra métricas de Prometheus y logs de MySQL. Se puede comparar un aumento de respuestas 500 con los eventos de error almacenados.

MySQL

Tabla app_logs: mensajes escritos por cada ruta de la API.

Grafana

Dashboard Demo de observabilidad: métricas de Prometheus y logs de MySQL.

Made with GAMMA

Cierre: qué aprendimos y preguntas

- Una métrica resume numéricamente el comportamiento de la aplicación.
- Un log describe un evento concreto.
- Prometheus consulta y almacena las métricas expuestas por la API.
- MySQL almacena los logs de esta demo.
- Grafana consulta esas fuentes y las muestra en un dashboard.

MÉTRICA detecta una tendencia → **LOG** ayuda a investigar el evento

Pregunta para conversar: si aumenta el número de errores en el gráfico, ¿qué información buscaríamos en los logs para investigar la causa?

Para detener los servicios sin borrar los datos:

```
docker compose down
```

Para detenerlos y borrar también los datos guardados en los volúmenes:

```
docker compose down -v
```

Made with GAMMA